# 怎么改

基于 8 轮实测与 20 次调用的逐段计时 · 2026-11-02 · 芝士

## 一、结论

**慢不是单次调用慢，是调用次数多。**每次调用 7 秒左右，一轮几十到上百次，于是十几分钟。
现在每次出第一个字中位 **1.73 秒**，多数不到 2 秒；请求发出前的准备几乎都在 0.4 秒以内。也就是说，人看到的「卡住」不是某一次特别慢，是很多次正常的等待叠起来。
所以改法的靶子只有一个：**把次数降下来**。下面三条都指向它，按性价比排。

## 二、依据

| 轮次总时长 | 调用次数 | 每次调用平均 | 首字出现 | 请求前准备 |
|---|---|---|---|---|
| 22.6s | 6 | 3.8s | 3.31s | 1.79s |
| 22.2s | 7 | 3.2s | 1.01s | 0.07s |
| 72.6s | 10 | 7.3s | 6.15s | 1.87s |
| 136.1s | 20 | 6.8s | 0.96s | 0.07s |
| 277.2s | 29 | 9.6s | 3.36s | 0.35s |
| 618.5s | 79 | 7.8s | 1.31s | 0.07s |
| 814.0s | 119 | 6.8s | 8.57s | 0.12s |
| 1011.3s | 58 | 17.4s | 2.52s | 0.07s |

8 轮真实会话，2026-11-01 下午。合计 2,975 秒、328 次调用。

## 三、三条改法

### 1. 能并行的调用别串着发

**根因**：每次工具调用都要一次完整的模型往返，约 7 秒。三次独立的读文件、跑命令，串着发就是 21 秒，并行发还是 7 秒。一轮 79 次、119 次调用的那两轮，绝大部分时间就花在这里。
**改法**：一轮里把互不依赖的调用一起发出去。这是三条里唯一直接作用于「每次 7 秒 × 次数」这一项的。

### 2. 回复拦截不要用「撞墙」来实现

**根因**：平台规定「有人说话还没回时，除回话外的工具一律拒绝」。如果先动手再回话，那次工具调用就白挨一次拒绝，模型得重来一遍。
**改法**：把「回一句」变成提交前的一次检查，而不是事后撞墙；或者允许读类操作先跑。省的是纯浪费。

### 3. 提醒插在一轮中间，也占一次往返

**根因**：提醒的阈值是 600 秒沉默。核实过三次：它都**插进正在跑的那一轮**，不是新开一轮——所以成本是一次额外往返加一句回话，不是「整套开场重新付一遍」。
**改法**：看板上的状态不靠叫醒会话也能更新。这条省的是小钱，排在第三。

## 四、不该动的

**压上下文**：零思考调用的等待与上下文体量无关（回归 $r^2$=0.01），压缩它不省时间。
**改缓存结构**：实测每次已经在读 38–46 万 token 的缓存，命中很充分，没有可捡的。
**追长尾**：超过 30 秒的调用只占 3.3%，成因还没查清，硬改是猜。

## 五、这些结论的边界

第 2、3 条有机制和现场记录支撑。第 1 条是按 7 秒/次推算的，**没有做对照实验**——真要做，得拿同一件事串行跑一遍、并行跑一遍，比较总时长。
工具链那条（沙箱里没有编译器）单独写在了另一份里，不在这三条之内。

口径：8 轮会话为 2026-11-01 下午；首字延迟 20 次调用实测，中位 1.73 秒、范围 1.31–8.83 秒。